Week 4 **Order and Order Details Management System**

Scenario

AURA REALM is an online beauty platform where customers can browse and purchase various cosmetic and skincare products. The system needs to maintain information about customer orders and the individual beauty products included in each order.

The Orders table stores the overall information about an order, such as the customer who placed the order, order date, total amount, and order status. The OrderDetails table stores the details of the products purchased in each order, including the product, quantity, and price.

The Order and Order Details Management System helps AURA REALM maintain order information accurately and establish relationships between customers, orders, and beauty products.

1. Design Orders and Order Details Tables

Orders Table

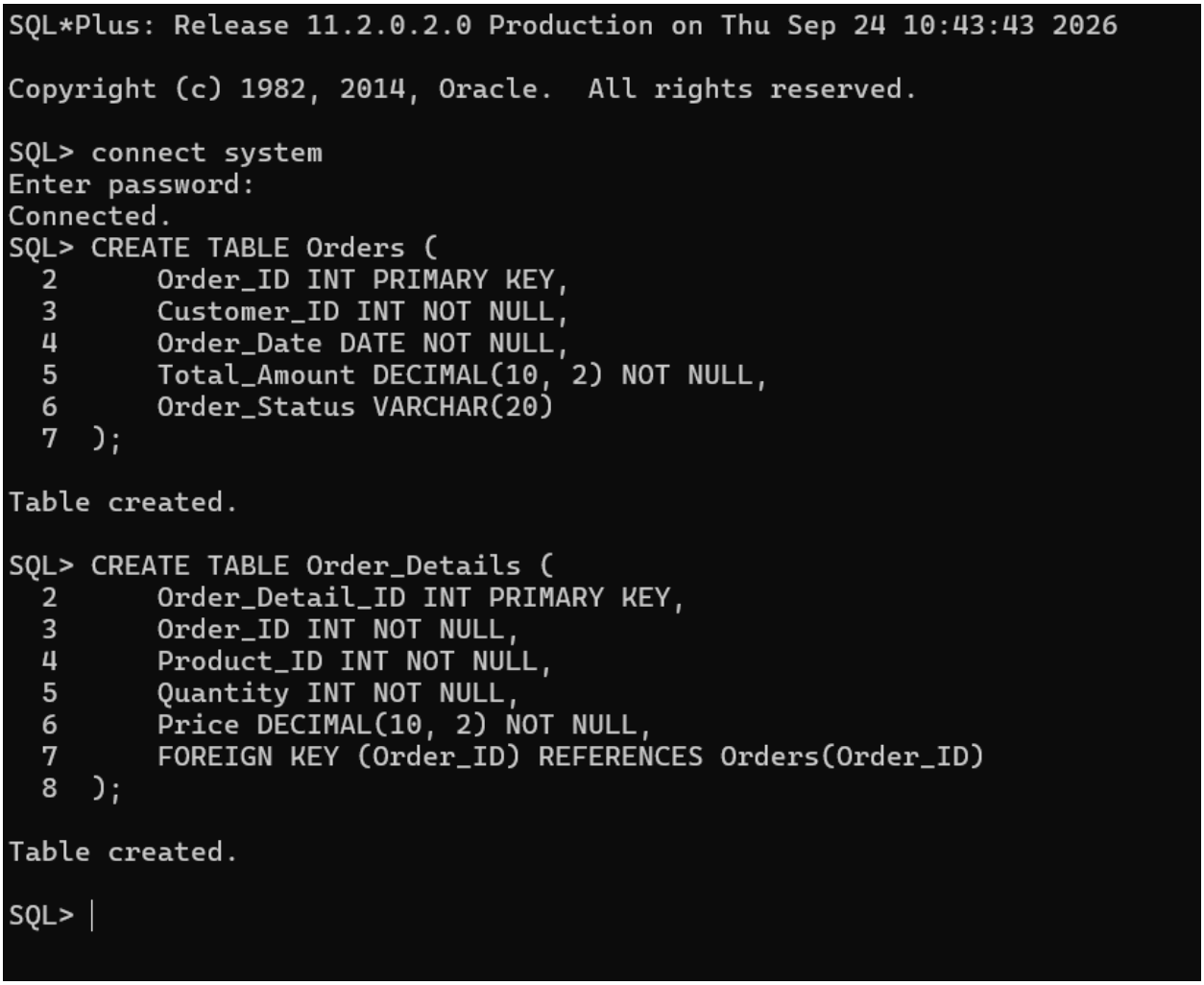
The Orders table is created to store the main information related to customer orders. The table contains the following attributes:

- **Order_ID** - Unique identifier for each order.
- **Customer_ID** - Identifies the customer who placed the order.
- **Order_Date** - Stores the date on which the order was placed.
- **Total_Amount** - Stores the total amount of the order.
- **Order_Status** - Stores the current status of the order.

OrderDetails Table

The OrderDetails table stores information about individual beauty products included in an order. The table contains:

- **Order_Detail_ID** - Unique identifier for each order detail.
- **Order_ID** - Identifies the corresponding order.
- **Product_ID** - Identifies the beauty product purchased.
- **Quantity** - Stores the quantity of product ordered.
- **Price** - Stores the price of the product.



2. Establish Relationships

Customer and Orders

One customer can place multiple orders, while each order belongs to one customer.

- **Relationship:** Customer → Orders = One-to-Many
- The Customer_ID in the Orders table is used to establish this relationship.

Orders and OrderDetails

One order can contain multiple order details, while each order detail belongs to one order.

- **Relationship:** Orders → OrderDetails = One-to-Many
- The Order_ID in OrderDetails connects each order detail with its corresponding order.

Product and OrderDetails

The OrderDetails table is connected with the Product table through Product_ID. This identifies the beauty product purchased in each order.

3. Apply Constraints

Primary key constraints were applied to uniquely identify records in both tables:

- **Orders:** Order_ID is the Primary Key.
- **OrderDetails:** Order_Detail_ID is the Primary Key.

Foreign key constraints were established to maintain relationships between the tables:

- Customer_ID in Orders references the Customer table.
- Order_ID in OrderDetails references the Orders table.

NOT NULL constraints were applied to important fields to ensure that required information is not left empty. CHECK constraints were also applied to maintain valid data:

- Order quantity must be greater than zero (> 0).
- Product price must be greater than zero (> 0).
- Total order amount must be greater than or equal to zero (≥ 0).

PRIMARY KEY CONSTRAINT:

```
SQL> -- Primary Key Constraints
SQL> ALTER TABLE Category
  2  ADD CONSTRAINT PK_Category PRIMARY KEY (Category_ID);

Table altered.

SQL>
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT PK_Product PRIMARY KEY (Product_ID);

Table altered.
```

FOREIGN KEY CONSTRAINT:

```
SQL>
SQL> -- Foreign Key Constraint
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT FK_Product_Category
  3  FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID);

Table altered.
```

NOT NULL CONSTRAINT:

```
SQL> -- NOT NULL Constraints
SQL> ALTER TABLE Category
  2  MODIFY Category_Name VARCHAR2(30) NOT NULL;

Table altered.

SQL>
SQL> ALTER TABLE Product
  2  MODIFY Product_Name VARCHAR2(50) NOT NULL;

Table altered.
```

CHECK CONSTRAINT:

```
SQL> -- CHECK Constraints
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT CHK_Product_Price CHECK (Price > 0);

Table altered.

SQL>
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT CHK_Stock CHECK (Stock_Quantity >= 0);

Table altered.
```

UNIQUE CONSTRAINT:

```
SQL> -- UNIQUE Constraint
SQL> ALTER TABLE Category
  2  ADD CONSTRAINT UQ_Category_Name UNIQUE (Category_Name);

Table altered.
```